



Code Quality Studio

The Complete Handbook

530 rules

18 stacks

CLI · Web · MCP · Agents · CI

Everything in one place — install it, run it locally, wire it into CI, drive it from an AI assistant, and let the agents fix things for you.

Version 2.3.1

`npm install -g cqs-audit`

Contents

1. What this is

1. The problem it solves
2. How analysis actually works
3. The five ways to run it

2. Installing

1. Prerequisites
2. Install from npm
3. Staying up to date
4. The nvm trap
5. Building from source

3. Running locally — the CLI

1. Your first audit
2. Reading the output
3. Filtering
4. Output formats
5. Every option

4. The report

1. What you get
2. The seven views
3. Sharing it
4. Controlling when it appears

5. The web app

1. Running it
2. Loading files
3. What the app adds

6. Using it from an AI assistant — MCP

1. What MCP is
2. The seven tools
3. Setup per client
4. Verifying
5. Talking to it

7. The agents

1. Remediation agent
2. PR review bot
3. Providers and keys
4. Safety model

8. CI integration

1. Exit codes
2. GitHub Actions
3. Other CI systems
4. Pre-commit hooks

9. Custom rules

1. The rules file
2. Writing a rule
3. Testing before committing

10. Reference

1. All 18 stacks
2. Categories and scoring
3. Troubleshooting
4. FAQ

1 · What this is

1.1 The problem it solves

Test suites rot quietly. A `waitForTimeout(5000)` added to unblock a release survives for three years. An XPath locator copied from the browser devtools breaks the moment a designer touches the markup. A test with no `expect()` at all passes forever and proves nothing.

None of this shows up as a failure. The suite stays green while slowly becoming worthless. Code Quality Studio finds these patterns and tells you which ones matter, with a fix for each.

1.2 How analysis actually works

This is the single most important thing to understand, because it explains both the strengths and the limits.

The rules are pattern matching, not compilation. There is no AST, no type checker, no language server. Each rule is a regular expression plus context — what line it matched, what block it sits inside, what else the file contains.

This buys you	This costs you
18 languages and frameworks with one engine. No toolchain per language, no <code>tsc</code> , no JVM, no Python runtime.	No type-aware checks. It cannot tell you a variable is the wrong type.
Runs in a browser, a CLI, and an MCP server from the same source.	Occasional false positives where a pattern appears in a context the rule did not anticipate.
Milliseconds per file. A 76-file suite audits faster than a test starts.	Cannot follow a value across files.
Zero dependencies to install per stack.	Rules need care: each one is verified to fire on the defect <i>and</i> stay silent on correct code.

Why this matters to you: treat findings as *informed suggestions*, not compiler errors. The critical ones are almost always real. If a rule misfires on your codebase, turn it off in Rule Settings — that preference is remembered.

1.3 The five ways to run it

Way	Best for	Chapter
CLI — <code>cqs ./tests/</code>	Daily use. Fastest path from question to answer.	3
Report — one HTML file	Sharing with people who will not run a terminal.	4
Web app — browser UI	Exploring interactively, toggling rules, re-running.	5
MCP — from your AI assistant	Asking in plain English; letting the assistant interpret.	6
Agents — remediate, pr-review	Fixing at scale; reviewing pull requests.	7

All five share one engine. The same 530 rules produce the same findings regardless of how you invoke them.

2 · Installing

2.1 Prerequisites

Node.js 18 or newer. Nothing else.

```
node --version      # expect v18.0.0 or higher
```

If that says "command not found", install Node from nodejs.org first.

2.2 Install from npm

```
npm install -g cqs-audit
```

`-g` means "available everywhere on this machine". Confirm:

```
cqs --help
```

You should see the help text with a version number at the top.

2.3 Staying up to date

Goal	Command
Upgrade to the newest release	<code>npm install -g cqs-audit@latest</code>
See the version you have	<code>cqs --help head -2</code>
See the newest published	<code>npm view cqs-audit version</code>
See every published version	<code>npm view cqs-audit versions</code>
Install one specific version	<code>npm install -g cqs-audit@2.3.0</code>
Remove it	<code>npm uninstall -g cqs-audit</code>

npm's web page lags. npmjs.com caches package pages and can show an older version for up to an hour after a release. `npm view` queries the registry directly and is authoritative — it is the same source `npm install` uses. Trust the command, not the page.

2.4 The nvm trap

This catches nearly everyone who uses `nvm`, so it gets its own section.

Global npm packages are installed per Node version. Install `cqs` while on Node 22, switch to Node 24, and `cqs` either vanishes or silently runs an older copy you installed months ago.

```
~/ .nvm/versions/node/  
├─ v22.22.0/  
│   └─ lib/node_modules/cqs-audit/ ← an old version, still here  
└─ v24.20.0/  
    └─ lib/node_modules/cqs-audit/ ← the one you just installed
```

Which one runs depends on ``nvm use``, not on which is newer.

Diagnose it:

```
node --version      # which Node am I on?  
which cqs           # which binary will run?  
type -a cqs         # every cqs on the PATH  
cqs --help | head -2 # what version is it really?
```

If they disagree, reinstall under the version you actually use:

```
npm install -g cqs-audit@latest
```

2.5 Building from source

Order matters here. The CLI inlines the built web app so it can emit reports, so **the web app must be built first**.

```
# from a checkout of the repository  
cd code-quality-studio  
  
npm install  
npm run build      # builds the web app into dist/assets – do this first  
  
cd cli  
npm install        # installs esbuild  
npm run build:all  # bundles the CLI and the MCP server  
npm install -g .    # registers `cqs` globally
```

Skip `npm run build` at the root and the CLI still builds — it just prints a warning and reports fall back to a flat HTML document instead of the full app. If your reports look plain, this is why.

What gets built

Artefact	Size	Contains
cli/dist/cqs.js	~1.5 MB	CLI + 530 rules + the inlined web app
cli/dist/cqs-mcp.js	~1.3 MB	MCP server + the same rules and app
dist/assets/	~830 KB	The built web app the other two embed

3 · Running locally — the CLI

3.1 Your first audit

```
cqs ./tests/
```

That is the whole thing. It detects your framework from the file extensions, applies the matching rules, prints findings, and opens a full report in your browser.

No flags needed for the common case. No config file. No API key.

3.2 Reading the output

Findings come in three severities, and the distinction is deliberate:

Severity	Means	Example
CRITICAL	This will bite you. Flaky, insecure, or proving nothing.	No <code>expect()</code> in the test; hardcoded credentials
WARNING	Works today, fragile tomorrow.	XPath locator; <code>waitForTimeout</code>
INFO	Worth knowing, not worth blocking on.	Missing tags for selective runs

Each finding carries a rule ID such as `PW-REL-001`. The prefix is the stack, the middle is the category, the number identifies the rule. Use it to look a rule up or to disable it.

3.3 Filtering

```
cqs ./tests/ --severity critical    # only what must be fixed
cqs ./tests/ --severity warning    # only warnings
cqs ./tests/ --category reliability # only flakiness
cqs ./tests/ --category selectors  # only locator problems
```

Filters apply to every output mode, including `summary`.

Filtering never hides a failing build. The exit code counts critical findings across everything, even when you filter the display to something else. Narrowing what you look at cannot accidentally make CI pass.

3.4 Output formats

pretty (default)

Coloured terminal output, plus the browser report.

summary — one line, ideal for CI logs

```
cqs ./tests/ --output summary
cqs 🤖 Playwright (TS / JS) · 76 files · score 90 · 288 critical · 60 warning
```

json — machine-readable

```
cqs ./tests/ --output json > report.json
cqs --read-report report.json          # summarise it later
```

Shape: `{ stack, files, avgScore, summary, results[] }`, where each result has `file`, `overallScore`, `categoryScores` and `findings[]`.

3.5 Every option

Option	What it does
<code>-s, --stack <id></code>	Force a stack instead of auto-detecting
<code>-S, --severity <level></code>	<code>all</code> · <code>critical</code> · <code>warning</code> · <code>info</code>
<code>-c, --category <id></code>	Filter to one category
<code>-o, --output <fmt></code>	<code>pretty</code> · <code>json</code> · <code>summary</code>
<code>--open</code>	Force the report even when piped or in CI
<code>--no-report</code>	Skip the report for this run
<code>--rules <file></code>	Use a specific <code>cqs-rules.json</code>
<code>--no-rules</code>	Ignore any project rules file
<code>--no-color</code>	Disable ANSI colours
<code>--list-stacks</code>	Print all stack IDs and exit
<code>-r, --read-report <file></code>	Summarise a saved JSON report
<code>-h, --help</code>	Show help

4 · The report

4.1 What you get

When `cqs` finishes it writes **one HTML file containing the entire Code Quality Studio application** — the React app, its styles, and your results, inlined together.

```
cqs ./tests/
├── terminal output ..... findings, grouped by severity
└── one .html file ..... the whole app, ~900 KB
    ├── no server needed
    ├── no internet needed
    └── email it; it works on their machine
```

This is not a printout of the app. It *is* the app, with the results attached as a saved workspace.

4.2 The seven views

View	Shows
 Overview	Score, verdict, counts, what to fix first
 Files	Every file as a card with per-category bars
 Findings	Each finding: the problem, why it matters, your code vs the fix
 Rule Settings	All rules for the stack, searchable, with toggles
 Coverage Radar	Category scores as a shape — weak areas are visible instantly
 Roadmap	Three phases: Day 1, Week 1, Month 1
 Practices	The best-practice checklist for your stack

4.3 Sharing it

The file is self-contained, so it survives being emailed, dropped in Slack, or attached to a ticket. The recipient needs a browser and nothing else.

Controls that cannot work offline are hidden. The report carries results but not your source files, so Run, Open, re-run, the AI providers, the stack picker and the rules terminal are removed. They would fail if clicked rather than merely be unused. What remains is every view and all the data.

4.4 Controlling when it appears

Command	Report?	Why
<code>cqs ./tests/</code>	Yes	Default — it is the point of the tool
<code>cqs ./tests/ --no-report</code>	No	You asked not to
<code>cqs ./tests/ --output json</code>	No	Machine-readable output; a browser would be wrong
<code>cqs ./tests/ --output summary</code>	No	Same
<code>cqs ./tests/ tee log</code>	No	Not a terminal — probably a pipeline
<code>cqs ./tests/ --open</code>	Yes	<code>--open</code> overrides all of the above

5 · The web app

5.1 Running it

```
# from a checkout of the repository
cd code-quality-studio
npm install
npm run dev                # http://localhost:5173
```

5.2 Loading files

Use + **Open** in the header, then either *Load test files* for individual files or *Load folder* for a directory. The browser asks permission once per folder.

Files are read in your browser and never uploaded. The analysis runs client-side — this is why it works with no backend.

5.3 What the app adds over the CLI

Capability	Why it needs the app
Toggling rules on and off	Preferences persist in the browser and apply to later runs
Re-running after an edit	The app keeps file contents; the report does not
Adding custom rules through a form	Writes your <code>cqs-rules.json</code> for you
AI review side by side	Compare rule findings with an AI second opinion per file
Duplicate detection across files	Cross-file analysis currently runs in the app only

For everything else — scores, findings, radar, roadmap — the CLI report gives you identical content.

6 · Using it from an AI assistant (MCP)

6.1 What MCP is

MCP (Model Context Protocol) lets an AI assistant use tools on your computer.

By default an assistant can only talk — it cannot read your files or run your programs. MCP is the socket you plug a tool into. Think of the assistant as a skilled colleague on a phone call: they can advise, but they cannot touch your laptop. MCP is you sharing the screen and handing over the keyboard, for one specific program.



Property	What it means for you
Runs locally	Your code never leaves the machine
Narrow by design	cqs can audit and write reports. It cannot delete, email, or browse
You stay in control	The assistant asks before running a tool; unplug it any time

6.2 The seven tools

You never name these. You ask in ordinary language and the assistant picks.

Tool	Does	You would say
<code>cqs_audit</code>	Finds problems	"What's wrong with my tests?"
<code>cqs_report</code>	Writes the full visual report	"Give me a report I can share"
<code>cqs_list_stacks</code>	Lists the 18 frameworks	"What does this support?"
<code>cqs_list_rules</code>	Lists rules for a stack	"What does it check for Cypress?"
<code>cqs_validate_rules</code>	Checks a custom rules file	"Is my rules file valid?"
<code>cqs_test_rule</code>	Tries a rule before committing	"Would this rule catch anything?"
<code>cqs_read_report</code>	Summarises a saved JSON report	"Summarise last week's report"

6.3 Setup

Step 1 — find the server file

```
echo "$(npm root -g)/cqs-audit/dist/cqs-mcp.js"
```

Copy the path it prints. It differs per machine — use yours, not an example.

Step 2a — Claude Code (easiest)

```
claude mcp add cqs node "$(npm root -g)/cqs-audit/dist/cqs-mcp.js"
```

Step 2b — Claude Desktop

Settings → Developer → Edit Config, or edit `~/Library/Application`

`Support/Claude/claude_desktop_config.json`:

```
{
  "mcpServers": {
    "cqs": {
      "command": "node",
      "args": ["/your/path/from/step/1/cqs-mcp.js"]
    }
  }
}
```

Then **quit the app completely** (Cmd+Q — closing the window is not enough) and reopen. Config is read only at startup.

If the file already has content, do not replace it. Add `"cqs": { ... }` inside the existing `"mcpServers"` block with a comma between entries. One misplaced comma makes the JSON invalid, and then *no* servers load — not just this one.

Step 2c — Cursor

Same JSON shape in `~/.cursor/mcp.json`, then restart Cursor.

6.4 Verifying

Ask the assistant: *"What cqs tools do you have?"* — it should list seven. If not, test the server directly, which rules out the client entirely:

```
printf '%s\n' '{"jsonrpc":"2.0","id":1,"method":"initialize","params":{"protocolVersion":"2024-11-05","capabilities":{},"clientInfo":{"name":"t","version":"1"}}}'
'{"jsonrpc":"2.0","id":2,"method":"tools/list"}' | node "$(npm root -g)/cqs-audit/dist/cqs-mcp.js"
```

JSON mentioning `cqs_audit` means the server is healthy and the problem is your config or the restart.

Check the config parses:

```
python3 -m json.tool ~/Library/Application\ Support/Claude/claude_desktop_config.json
```

6.5 Talking to it

```
"Audit ./tests and tell me the three worst problems."
"Generate a cqs report for ./tests and give me the path."
"Which files have no assertions?"
"What rules does cqs check for Playwright reliability?"
"Would a rule matching /page\\.\\$\\$ / catch anything in ./tests?"
```

7 · The agents

Everything so far *reports*. The two agents *act*. Both need an AI provider key; neither is required to use the tool.

7.1 Remediation agent — bulk auto-fix

```
cqs remediate ./tests/ --ai anthropic --dry-run
```

It audits, sends each critical finding plus its surrounding code to the model, and asks for a corrected version. The interesting part is what happens next.



A fix is never trusted because a model produced it. Every edit is re-run through the same 530 rules, and an edit that introduces a new critical is thrown away. A suspicious shrink — a model that "fixes" a file by deleting most of it — is caught too.

Flag	Effect
<code>--dry-run</code>	Show the diff, write nothing. Start here.
<code>--ai <provider></code>	<code>anthropic</code> · <code>openai</code> · <code>bedrock</code> · <code>vertex</code>
<code>--api-key <key></code>	Or use the provider's env var
<code>--model <id></code>	Override the default model
<code>--max-files <n></code>	Bound the run (default 10)
<code>--branch <name></code>	Create a branch before committing
<code>--commit</code>	Commit the accepted fixes
<code>--force</code>	Proceed despite a dirty working tree

Run it on a clean checkout. Without `--dry-run` it edits files in place. It refuses to start on a dirty tree unless you pass `--force` — that guard exists so your own uncommitted work never gets mixed into a machine-generated diff.

A sensible progression:

```
cqs remediate ./tests/ --ai anthropic --dry-run          # look first
cqs remediate ./tests/ --ai anthropic --max-files 3      # small, reviewable
cqs remediate ./tests/ --ai anthropic --branch fix/cqs --commit
```

7.2 PR review bot — inline GitHub comments

```
cqs pr-review --repo owner/name --pr 42 --token "$GITHUB_TOKEN" --ai anthropic
```

It fetches the pull request's changed files, audits them, and posts findings as **inline review comments on the exact lines** — not one wall of text at the bottom.

Getting that right is fiddlier than it sounds. GitHub only accepts a comment on a line that appears in the diff, addressed by its position within the diff hunk rather than its line number in the file. The bot parses the @@ -a,b +c,d @@ headers to work out which lines are commentable; anything outside them is collected into the review summary instead of being silently dropped.

Flag	Effect
--repo <owner/name>	Target repository
--pr <number>	Pull request number
--token <token>	GitHub token with PR write access
--threshold <n>	Request changes above this many criticals
--max-comments <n>	Cap the comments (default 30)
--only-added	Comment only on added lines, not context

Why the cap matters. A PR that touches a legacy file can produce hundreds of findings. Thirty inline comments is a review; three hundred is noise that gets the bot muted.

7.3 Providers and keys

Provider	Flag	Environment variable
Anthropic	--ai anthropic	ANTHROPIC_API_KEY
OpenAI	--ai openai	OPENAI_API_KEY
AWS Bedrock	--ai bedrock	--aws-region , --aws-access-key , --aws-secret-key
Google Vertex	--ai vertex	--vertex-project , --vertex-location , --vertex-key-file

Keys are only ever needed for the agents and the optional AI review. All 530 rules run locally with no key, no network and no account. If someone tells you the tool needs an API key to audit code, they are describing a different tool.

7.4 The safety model in one paragraph

Rules are deterministic and trusted. Model output is neither, so it is treated as a proposal: generated, re-audited, and kept only if the file got better by the same measure that flagged it in the first place. That is why the remediation agent can be pointed at a real branch without much anxiety — the worst case is that it declines to fix something.

8 · CI integration

8.1 Exit codes

Code	Meaning
0	No critical findings — or you filtered to <code>info</code> / <code>warning</code>
1	At least one critical finding, or the command failed

Criticals set the exit code even when the display is filtered to something else, so narrowing output cannot accidentally turn a red build green.

8.2 GitHub Actions

```
name: Test quality

on: [push, pull_request]

jobs:
  audit:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 22

      - run: npm install -g cqs-audit
      - run: cqs ./tests/ --output summary
```

Keep the JSON as an artefact so you can compare week to week:

```
- run: cqs ./tests/ --output json > cqs-report.json
  continue-on-error: true
- uses: actions/upload-artifact@v4
  with:
    name: cqs-report
    path: cqs-report.json
```

No `--no-report` needed in CI. Reports are skipped automatically when output is not a terminal, and `--output json` and `--output summary` never produce one.

Review pull requests automatically

```
review:
  if: github.event_name == 'pull_request'
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - run: npm install -g cqs-audit
    - run: |
        cqs pr-review \
          --repo ${github.repository} \
          --pr ${github.event.pull_request.number} \
          --token ${secrets.GITHUB_TOKEN} \
          --ai anthropic
  env:
    ANTHROPIC_API_KEY: ${secrets.ANTHROPIC_API_KEY}
```

8.3 Other CI systems

GitLab CI

```
test-quality:
  image: node:22
  script:
    - npm install -g cqs-audit
    - cqs ./tests/ --output summary
```

Jenkins

```
stage('Test quality') {
  steps {
    sh 'npm install -g cqs-audit'
    sh 'cqs ./tests/ --output summary'
  }
}
```

Azure Pipelines

```
- script: |
    npm install -g cqs-audit
    cqs ./tests/ --output summary
  displayName: 'Audit test quality'
```

8.4 Pre-commit hooks

Block only what is genuinely broken, or the hook gets bypassed within a week:

```
# .git/hooks/pre-commit  (chmod +x)
cqs . --severity critical --output summary || {
  echo "Critical test-quality issues. Fix them or commit with --no-verify."
  exit 1
}
```

Introducing this to an existing codebase? Start with `--severity critical` and nothing else. A hook that fires on 300 warnings on day one teaches everyone to use `--no-verify`, and then it protects nothing.

Appendix · What the report looks like

Screenshots from a report generated against a small fictional suite (`acme-shop` : three Playwright specs, deliberately flawed). Everything here is one HTML file opened in a browser — no server, no internet, no install.

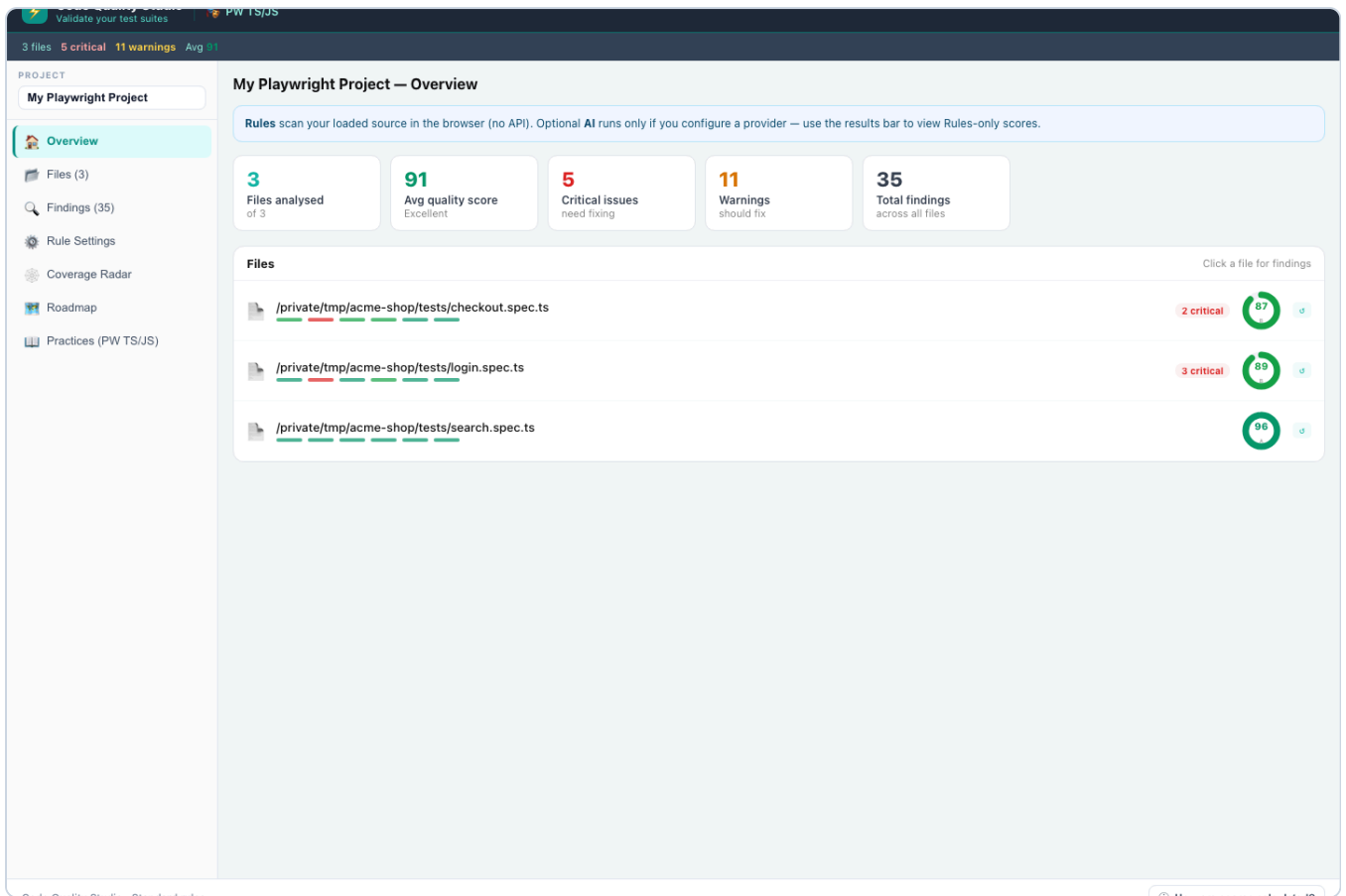
Files

Every spec as a card: overall score, and a bar per category showing what drove it. The count beside each bar is the number of findings in that category.



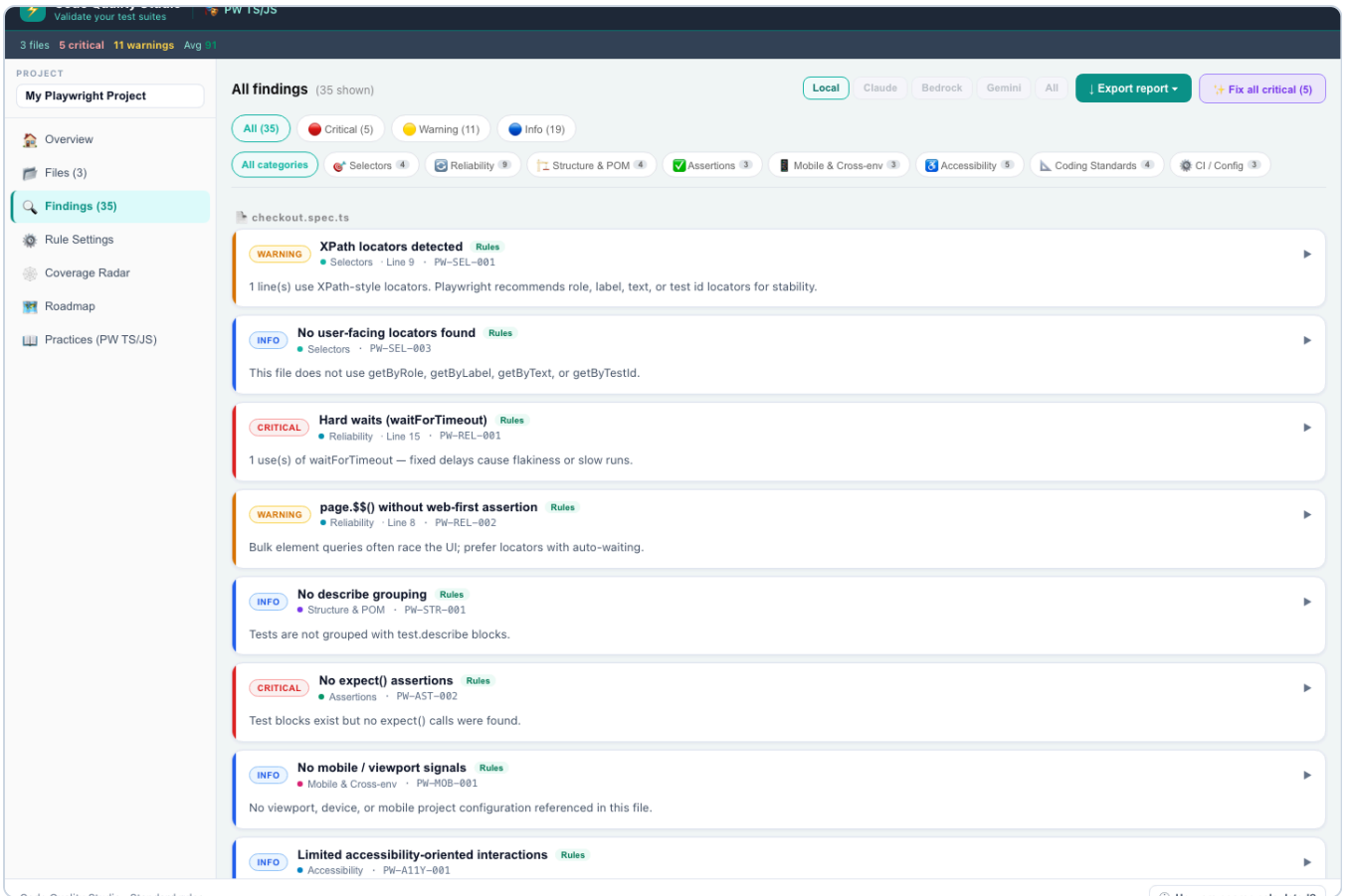
Overview

The headline verdict and where to start. Written to be read by someone who will not open the other views.



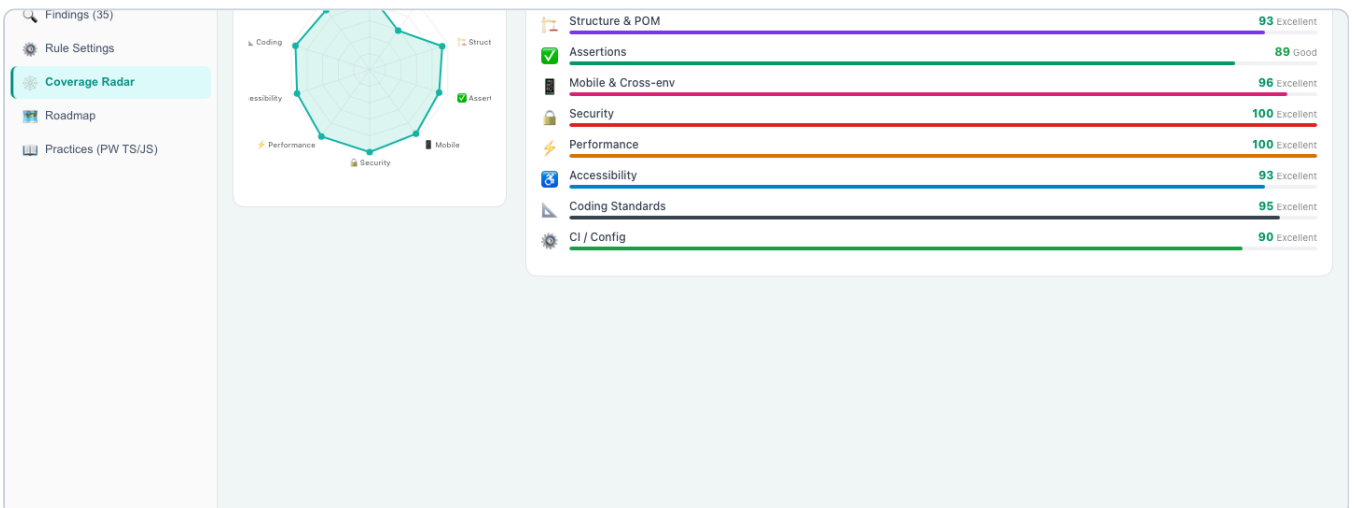
Findings

Each finding in full: the problem, why it matters in plain terms, your code beside the recommended fix, and a link to the reference.



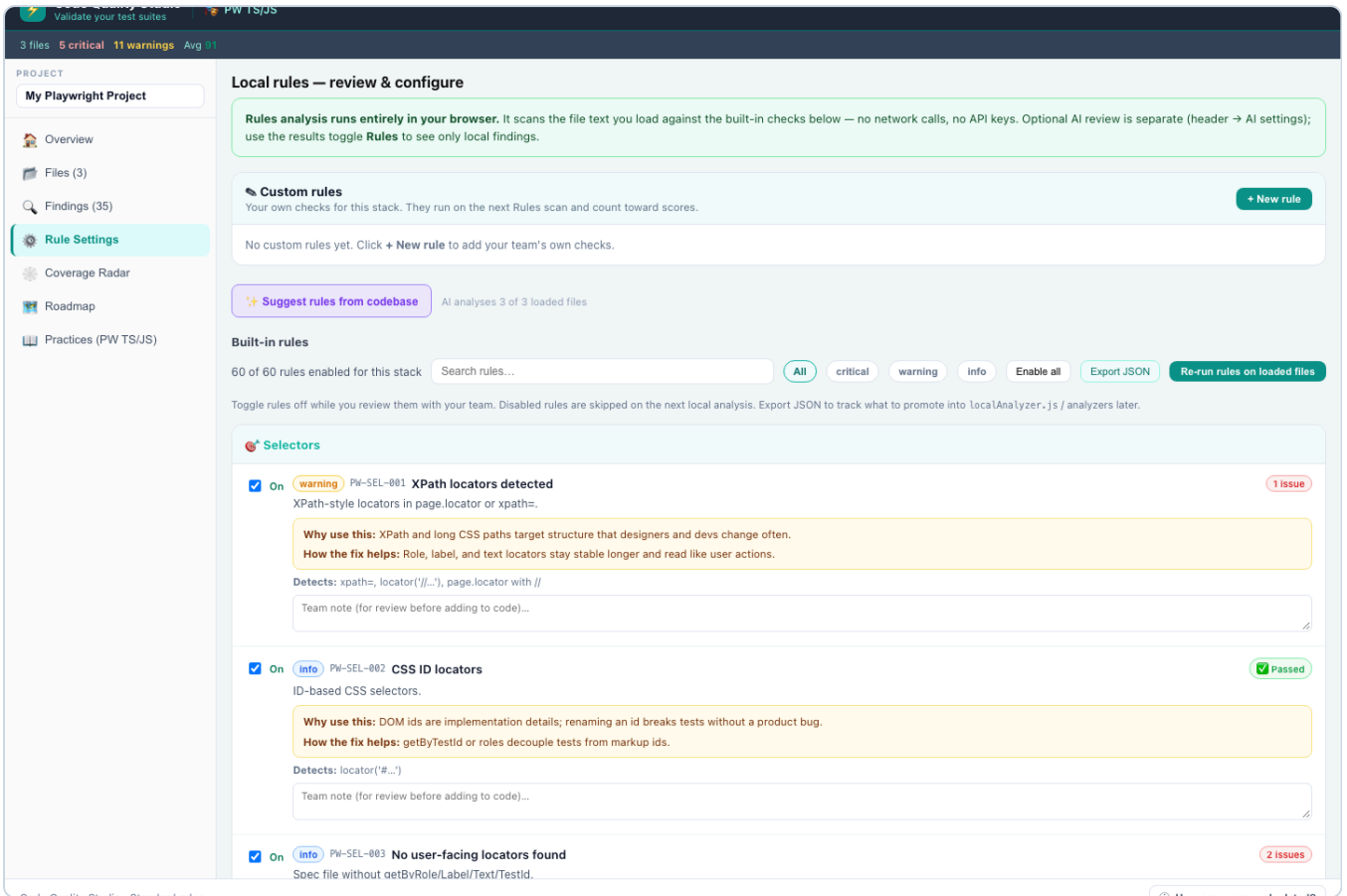
Coverage Radar

Category scores as a shape. Reliability at 59 is obvious here in a way that an overall score of 91 completely hides.



Rule Settings

Every rule for the stack, searchable, each one toggleable. Turn off what does not apply to your codebase; the choice persists.



Roadmap

The same findings sequenced into Day 1, Week 1 and Month 1, so a team knows what to do first.

Validate your test suites

PW TS/JS

3 files 5 critical 11 warnings Avg 81

PROJECT

My Playwright Project

Overview

Files (3)

Findings (35)

Rule Settings

Coverage Radar

Roadmap

Practices (PW TS/JS)

Fix roadmap — My Playwright Project

1 Immediate (Day 1)

5 findings

Hard waits (waitForTimeout)

checkout.spec.ts PW-REL-001 line 15

1 use(s) of waitForTimeout — fixed delays cause flakiness or slow runs.

Go to file

No expect() assertions

checkout.spec.ts PW-AST-002

Test blocks exist but no expect() calls were found.

Go to file

Hard wait (waitForTimeout) — occurrence 1/2

login.spec.ts PW-REL-001 line 5

Fixed delay at line 5 (2 total in file). Replace with a web-first assertion.

Go to file

Hard wait (waitForTimeout) — occurrence 2/2

login.spec.ts PW-REL-001 line 10

Fixed delay at line 10 (2 total in file). Replace with a web-first assertion.

Go to file

Missing await on async Playwright call

login.spec.ts PW-REL-009 line 16

Line 16 calls an async Playwright action without 'await'. The promise floats and the test continues before the action completes.

Go to file

2 Short-term (Week 1)

11 findings

XPath locators detected

checkout.spec.ts PW-SEL-001 line 9

1 line(s) use XPath-style locators. Playwright recommends role, label, text, or test id locators for stability.

Go to file

page.\$\$() without web-first assertion

checkout.spec.ts PW-REL-002 line 8

Bulk element queries often race the UI; prefer locators with auto-waiting.

Go to file

Hardcoded absolute URLs

checkout.spec.ts PW-CI-001 line 7

Use baseUrl in playwright.config and relative paths in tests.

Go to file

waitForSelector used

checkout.spec.ts PW-REL-003 line 10

Go to file

9 · Custom rules

9.1 The rules file

Drop a `cqs-rules.json` beside your tests. It is discovered by walking up from the audited path and stopping at `.git`, so one file at the repo root covers everything beneath it.

9.2 Writing a rule

```
{
  "rules": [
    {
      "id": "TEAM-001",
      "stack": "playwright",
      "severity": "warning",
      "category": "selectors",
      "pattern": "getByTestId\\(['\"]tmp-",
      "title": "Temporary test id left in a locator",
      "description": "tmp-* ids are scaffolding and get removed without warning.",
      "fix": "Replace with a stable role or label selector."
    }
  ]
}
```

Field	Required	Notes
id	yes	Unique. Prefix with your team, e.g. <code>TEAM-</code>
stack	yes	Which stack it applies to
pattern	yes	JavaScript regex, JSON-escaped
severity	yes	<code>critical</code> · <code>warning</code> · <code>info</code>
category	yes	An existing category id
title	yes	Shown in the report
description	no	Why it matters
fix	no	What to do instead

Backslashes are doubled in JSON. The regex `page\\.\\.\\$\\$` is written `"page\\\\.\\.\\$\\$"`. Getting this wrong is the most common reason a rule silently matches nothing.

9.3 Test before you commit

A rule that matches nothing is invisible; a rule that matches everything is worse. Check both before it reaches your team. Over MCP, ask your assistant:

```
"Validate the cqs rules file in this repo."  
"Would a rule matching page\\.\\$ catch anything in ./tests?"
```

Or from the CLI:

```
cqs ./tests/ --rules ./cqs-rules.json      # use a specific file  
cqs ./tests/ --no-rules                    # ignore project rules entirely
```

The test that matters: does it fire on the defect, *and* stay silent on code you consider correct? A rule that only passes the first half will be turned off by the first person it annoys.

10 · Reference

10.1 All 18 stacks

ID	Stack	Files	Categories
playwright	Playwright (TS/JS)	.ts .js .tsx .jsx	10
playwright_java	Playwright (Java)	.java	9
playwright_python	Playwright (Python)	.py	8
selenium_java	Selenium (Java)	.java	8
selenium_csharp	Selenium (C#)	.cs	8
cypress	Cypress (TS/JS)	.ts .js	8
appium_java	Appium (Java)	.java	8
tosca_xml	TOSCA (XML export)	.xml	8
restassured	REST Assured (Java)	.java	6
karate	Karate (API DSL)	.feature	6
pytest_api	Python API (pytest)	.py	7
postman	Postman / Newman	.json	4
java_api	Java API / Core	.java	10
java_frontend	Java · Core / General	.java	5
typescript	TypeScript Core / API	.ts	10
ts_frontend	TypeScript · Frontend (React)	.tsx .ts	6
python_api	Python · API / Core	.py	8
python_frontend	Python · Frontend (Templates)	.html	3

10.2 Categories and scoring

Playwright's ten categories, as an example — other stacks use a subset:

Category	Catches
 Selectors	XPath, CSS ids, index-based locators, brittle chains
 Reliability	Hard waits, missing awaits, conditional flow, networkidle
 Structure & POM	Page objects, fixtures, shared state, serial coupling
 Assertions	Missing or weak assertions, non-native matchers
 Mobile & Cross-env	Viewports, device projects, hardcoded sizes
 Security	Credentials in code, disabled TLS, unsafe browser flags
 Performance	Sleeps in loops, redundant navigation
 Accessibility	Role-based locators, keyboard paths, a11y scans
 Coding Standards	Naming, file size, argument counts, leftover logs
 CI / Config	baseUrl, retries, tracing, <code>.only</code> , <code>headless:false</code>

How the score is calculated

Each file starts at 100 and loses points per finding, weighted by severity — criticals cost most. Category scores work the same way within their category. The suite score is the mean across files, so one terrible file in a hundred barely moves it, while a systemic problem does.

Read the categories, not the total. A suite scoring 90 overall with Reliability at 59 has a flakiness problem that the headline number hides. The radar exists for exactly this reason.

10.3 Troubleshooting

Symptom	Cause and fix
<code>command not found: cqs</code>	Installed under a different Node version. <code>node --version</code> , then reinstall.
Old version running after upgrade	nvm per-version globals — see §2.4.
Report looks plain, no sidebar	CLI built without the web app. Run <code>npm run build</code> at the repo root, then rebuild the CLI.
Browser shows an old report	<code>--open</code> writes a new timestamped file each run; macOS may re-focus an old tab. Close old tabs, or open the printed path.
Assistant cannot see the MCP tools	Fully quit and reopen the client; verify the path; validate the JSON.
No files found	Wrong stack detected. Force it: <code>--stack playwright</code> .
A rule misfires on your code	Turn it off in Rule Settings; the preference persists.
<code>remediate</code> refuses to run	Dirty working tree. Commit, stash, or pass <code>--force</code> .

10.4 FAQ

Does my code leave the machine?

No. Rules are local pattern matching. Only the two agents and the optional AI review contact a provider, and only when you pass `--ai`.

Do I need an API key?

Not for auditing, reports, MCP, or CI. Only for the agents.

Is the CLI report the same as the web app?

Yes — it is the same application, inlined into one file with your results attached. Controls that need source files are hidden because they cannot work offline.

Are MCP and CLI results the same?

Identical. Same engine, same rules, and `cqs_report` produces byte-identical HTML to `cqs --open` for the same input.

Can I use it on application code, not just tests?

Yes. `java_api`, `typescript`, `ts_frontend`, `python_api` and others target production code.

How do I add my team's conventions?

A `cqs-rules.json` at the repo root — see chapter 9.

Code Quality Studio · 530 rules · 18 stacks · CLI, web, MCP, agents and CI from one engine.

```
npm install -g cqs-audit
```